



Design and Implementation of a Secure Core Banking System (SCBS)

Muhammad Ismail	2023452
Ali Ahmed Chaudhry	2023093
Shayan Sidiqqi	2023656

1. Introduction

Financial systems are prime targets for cyberattacks due to the sensitivity of transactional and personal data. Secure banking software must guarantee confidentiality, integrity, availability, and accountability while resisting common threats such as injection attacks, privilege escalation, and session compromise. This project proposes the design and implementation of a Secure Core Banking System (SCBS) that integrates security into architecture, development, and testing. The system simulates a realistic banking environment with controlled user roles, secure transaction processing, and structured threat mitigation. The focus is secure software engineering, not merely functional feature development.

2. Problem Statement

Most academic banking applications prioritize functionality while neglecting core security controls such as atomic transaction handling, strict access enforcement, input validation, and structured threat modeling. In real-world systems, such weaknesses can lead to financial loss and system compromise. This project addresses this gap by embedding security mechanisms directly into system design and validating them through formal threat modeling and testing.

3. System Overview

The SCBS will implement four roles: Customer, Teller, Manager, and Administrator, each operating under strict Role-Based Access Control (RBAC) and least-privilege principles. Core functionalities include account management, deposits, withdrawals, and fund transfers. Transfers will be processed using atomic database transactions to ensure consistency. A double-entry ledger model will record debit and credit entries for each transaction, with balances derived from ledger data to maintain integrity. The system will also include a rule-based fraud detection mechanism to flag abnormal or high-risk transaction patterns.

4. Architecture

The system follows a layered architecture:

- **Presentation Layer:** Web interface
- **Application Layer:** Business logic, authentication, authorization
- **Database Layer:** ACID-compliant relational storage
- **Security Layer:** RBAC enforcement, validation, logging, monitoring

Trust boundaries will be defined between client input, server processing, and database operations. All sensitive actions will be validated server-side.

5. Security Requirements

The system enforces core security principles:

- **Confidentiality:** Secure password hashing (bcrypt/Argon2) and protected communication.
- **Integrity:** Atomic transactions, double-entry ledger accounting, HMAC validation, and hash-chained audit logs.
- **Availability:** Rate limiting and account lockout mechanisms.
- **Authentication & Authorization:** Multi-factor authentication (OTP simulation), secure session handling, and strict server-side RBAC enforcement.

6. Risk Management and Threat Modeling

Risks such as SQL injection, XSS, privilege escalation, replay attacks, brute-force attempts, and data tampering will be identified and mitigated through prepared statements, strict validation, nonce-based request verification, and tamper-evident logging. Threat modeling will be conducted using the STRIDE framework with Data Flow Diagrams to identify trust boundaries and attack surfaces. Security controls will be directly mapped to identified threats.

7. Development and Testing Plan

Development will follow secure coding practices from the outset. After implementation, structured security testing will be performed, including simulated SQL injection, XSS, privilege escalation, replay attack, and brute-force scenarios. A secure code review will be conducted, and identified vulnerabilities will be remediated before final submission.

8. Expected Outcome

The final system will demonstrate a secure, role-based banking platform with atomic transaction handling, tamper-evident logging, fraud detection capabilities, and resistance against common web application attacks. Deliverables will include source code, architectural diagrams, threat model documentation, risk analysis, and security testing results.